EXPERT DETAILS

Heterogeneous Expert Architecture

Selection Rationale, Mathematical Formulation, and Financial Application

Author 1[1], Author 2[1]

[1]Organization Name
contact@org.com

Abstract. 본 문서는 금융 상품 추천을 위한 Progressive Layered Extraction (PLE) 아키텍처에서 사용되는 7개 이질적 전문가(Heterogeneous Expert)의 선정 근거, 수학적 배경, 금융 도메인 적용 방안을 상세히 기술한다. 각 전문가는 피처 상호작용(DeepFM), 시계열 분해(Temporal Ensemble), 계층 구조(Hyperbolic GCN), 위상 구조(PersLay/TDA), 협업 필터링(LightGCN), 인과 추론(Causal/NOTEARS), 분포 매칭(Optimal Transport)이라는 고유한 수학적 관점을 제공하며, 어떠한 단일 전문가도 다른 전문가의 역할을 대체할 수 없다. 추가로 adaTT(Adaptive Task-aware Transfer)를 통한 18개 태스크 간 동적 지식 전이 메커니즘과 11개 학문 분야에서 도출된 316차원 피처 엔지니어링 체계를 기술한다.

Keywords: Heterogeneous Experts, PLE, DeepFM, Mamba, Hyperbolic GCN, Persistent Homology, NOTEARS, Optimal Transport, adaTT, Financial AI

설계 vs 구현 참고. 본 문서는 풀뱅크 설계(734D)를 기준으로 작성되었습니다. 현재 Santander 벤치마크 ►► 현은 316D (12 feature groups)입니다.

DeepFM Expert — Feature Interaction

선정 근거 (Why DeepFM?)

금융 추천은 개별 피처가 아닌 피처 상호작용에 의존한다. 예를 들어 "30대 + 디지털 활용도 높음 + 높은 RFM"이 결합될 때 온라인 투자 전환율이 급등하는 패턴은 단일 피처만으로는 포착할 수 없다. 644개 피처에 대해 모든 pairwise interaction을 명시적으로 모델링하면 $O(n^2) = 207,046$개의 파라미터가 필요하며, 대부분은 sparse 데이터에서 안정적으로 추정할 수 없다.

FM의 low-rank factorization은 교차 파라미터를 $O(nk) = 10,304$개로 감소시키면서도 공유 잠재 벡터를 통해 관측되지 않은 피처 쌍에 대한 일반화를 가능하게 한다. Deep 컴포넌트는 FM의 2차 한계를 넘어 3차 이상의 암시적 고차 상호작용을 추가한다.

대안 비교 (Why Not Alternatives?)

Aspect	DeepFM	DCNv2	AutoInt (Transformer)
Interaction order	FM: 2nd + Deep: arbitrary	Cross: $(l+1)$-th + Deep	Self-attention: arbitrary
Parameters	~ 169K	~ 2.5M (cross alone)	$O(n^2)$ attention
Inference speed	Fastest (2–5x vs Attention)	Medium	Slowest
Field awareness	Yes (28 fields)	No (raw features)	Yes (per-feature)

Table 1: DeepFM vs. 대안 아키텍처 비교. BARS 벤치마크(2022–2024)에서 DeepFM은 중소규모 데이터셋에서 최저 추론 지연 시간과 함께 일관된 상위 성능을 보인다.

Wide&Deep (Google, 2016)과 달리 DeepFM은 FM과 Deep 간 임베딩을 공유하여 수동 cross-feature engineering을 제거한다. "구조적 효율성(FM) + 보편적 표현력(Deep)"의 조합을 제공한다.

수학적 배경 (Key Formulations)

FM 2nd-order interaction:

$$\hat{y} = w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle x_i x_j \tag{1}$$

Low-rank factorization:

$$W \approx VV^\top, \quad V \in \mathbb{R}^{n \times k} \tag{2}$$

이로써 파라미터가 $O(n^2)$에서 $O(nk)$로 감소한다. 프로젝트에서는 $n = 644$, $k = 16$을 사용한다.

FM Trick ($O(nk)$ computation):

$$\sum_{i<j} \langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle x_i x_j = \frac{1}{2} \sum_{f=1}^{k} \left[\left(\sum_{i=1}^{n} v_{i,f} x_i \right)^2 - \sum_{i=1}^{n} \left(v_{i,f} x_i \right)^2 \right] \tag{3}$$

Cross Network (DCNv2) layer:

$$\boldsymbol{x}_{l+1} = \boldsymbol{x}_0 \circ (\boldsymbol{x}_l W_l + \boldsymbol{b}_l) + \boldsymbol{x}_l \tag{4}$$

각 레이어가 1차수의 interaction을 추가하며, l개 레이어는 최대 $(l+1)$차 interaction을 포착한다. 레이어당 $2d$ 파라미터만 필요하다 (MLP의 $O(d^2)$ 대비).

28-Field Architecture

644D 정규화 피처를 28개 시맨틱 필드로 분할한다 (base 238D, multi-source 91D, extended 84D, domain 159D, multidisciplinary 24D, model-derived 27D, merchant 21D). 각 필드는 `nn.Linear(d_i, 16)`을 통해 균일한 16D 잠재 공간으로 투영된다. FM은 필드 레벨 임베딩 $[B, 28, 16]$에서 동작하고, Deep MLP는 flatten된 $[B, 448]$에서 동작한다.

Output: FM 16D + Deep 64D = 80D $\rightarrow$ output layer $\rightarrow$ 64D (PLE gate 입력).

금융 적용 (Financial Interpretation)

- 교차 필드 상호작용은 은행업에서 핵심이다: "30대 + 서울 + 높은 디지털 활용" $\rightarrow$ 온라인 투자 전환율 급등; "높은 RFM + 낮은 예금" $\rightarrow$ 신용 상품 추천.
- 28-field 설계는 inter-category FM interaction을 가능하게 한다: 64D 카테고리를 4$\times$16D로 분할하면 추가 파라미터 비용 없이 27개의 새로운 FM interaction 쌍이 생성된다.

입출력 사양 (I/O Specification)

Input	644D normalized feature tensor $[B, 644]$
Internal	28 fields $\times$ 16D embeddings $\rightarrow$ FM $[B, 28, 16]$ + Deep $[B, 448]$
Output	64D expert representation for PLE CGC Gate
Parameters	~ 169K

Table 2: DeepFM Expert 입출력 사양.

구현 참고사항 (Implementation Notes)

- PLE의 기본 경량 Shared Expert로 사용. DCNv2Expert는 고차 상호작용이 필요한 경우의 대안이다.

- FM과 Deep이 임베딩을 공유하므로 gradient가 양쪽 경로를 통해 동시에 흐른다.
- FocalLoss 적용 시 pre-activation logits를 전달해야 한다 (double-sigmoid 방지).

주요 참고문헌: Guo et al. (IJCAI 2017), Rendle (ICDM 2010), Wang et al. (KDD 2017, WWW 2021).

Temporal Ensemble Expert — Mamba + LNN + PatchTST

선정 근거 (Why Temporal Ensemble?)

정적 피처(나이, 평균 지출)는 거래 시퀀스의 주기성, 추세, 불규칙 이벤트 패턴이라는 시간적 차원을 버린다. 180일 지출 시퀀스를 월 평균으로 압축하면 주간 주기, 추세 방향, 이상 폭발 패턴이 소실된다.

모든 시계열은 $y(t) = T(t) + S(t) + R(t)$ (추세 + 계절성 + 잔차)로 분해되며, 단일 아키텍처로는 세 가지를 모두 최적으로 포착할 수 없다:

Model	Temporal Pattern	Mechanism	Complexity
Mamba (SSM)	Long-range trend	Selective State Space (S6)	$O(L)$ linear
LNN (ODE)	Irregular residuals	Adaptive time-constant ODE	$O(1)$ single step
PatchTST	Global periodicity	Patch-level self-attention	$O((L/P)^2)$

Table 3: Temporal Ensemble의 세 모델 – 시계열 분해 $T + S + R$에 대응.

대안 비교 (Why Not Single Model?)

Generation	Approach	Limitation
1st	ARIMA, Exponential Smoothing	선형 가정, 수동 differencing
2nd	LSTM, GRU	$O(L)$ 순차 병목, vanishing gradient
3rd	Transformer	$O(L^2)$ complexity, 약한 순서 인코딩
4th (ours)	SSM + ODE + Patch Transformer	모델 복잡성 (entropy monitoring으로 완화)

Table 4: 시계열 모델링 세대별 비교.

수학적 배경 (Key Formulations)

Mamba (Selective State Space Model)

Continuous SSM:

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}u, \quad y = \boldsymbol{C}\boldsymbol{x} + \boldsymbol{D}u \tag{5}$$

ZOH Discretization:

$$\bar{\boldsymbol{A}} = \exp(\Delta \cdot \boldsymbol{A}), \quad \bar{\boldsymbol{B}} \approx \Delta \cdot \boldsymbol{B} \tag{6}$$

Discrete recurrence:

$$\boldsymbol{h}_t = \bar{\boldsymbol{A}} \cdot \boldsymbol{h}_{t-1} + \bar{\boldsymbol{B}} \cdot \boldsymbol{x}_t, \quad \boldsymbol{y}_t = \boldsymbol{C}_t \cdot \boldsymbol{h}_t \tag{7}$$

S6 Selective Mechanism (핵심 혁신):

$$\Delta = \text{softplus}(\boldsymbol{W}_\Delta \cdot \boldsymbol{x} + \boldsymbol{b}_\Delta) \tag{8}$$

$$\boldsymbol{B} = \boldsymbol{W}_B \cdot \boldsymbol{x}, \quad \boldsymbol{C} = \boldsymbol{W}_C \cdot \boldsymbol{x} \tag{9}$$

LTI 시스템과 달리 S6는 Δ, $\boldsymbol{B}$, $\boldsymbol{C}$를 입력 의존적으로 만들어 content-aware processing을 가능하게 한다. Δ가 크면 현재 입력을 상태에 강하게 인코딩하고, 작으면 이전 상태를 보존한다.

금융 해석: 대규모 거래 $\rightarrow$ 큰 Δ (강하게 기억), 소규모 일상 구매 $\rightarrow$ 작은 Δ (배경 소음으로 빠르게 망각).

사양: Transaction Mamba: $d_{\text{model}} = 128$, $d_{\text{input}} = 16$, $d_{\text{state}} = 16$, $\text{seq_len} = 180$. Session Mamba: $d_{\text{model}} = 64$. HiPPO-style 대각선 초기화 $[-1, -2, ..., -N]$.

Liquid Neural Network (LNN)

Core ODE:

$$\frac{d\boldsymbol{h}}{dt} = \frac{-\boldsymbol{h} + f(\boldsymbol{x}, \boldsymbol{h})}{\tau(\boldsymbol{x}, \boldsymbol{h})} \tag{10}$$

여기서:

- $-\boldsymbol{h}$: leak/decay term (입력 없이 0을 향해 망각)
- $f(\boldsymbol{x}, \boldsymbol{h}) = \tanh\big(\boldsymbol{W}_f[\boldsymbol{x}; \boldsymbol{h}] + \boldsymbol{b}_f\big)$: target state
- $\tau(\boldsymbol{x}, \boldsymbol{h}) = \text{Softplus}(\text{MLP}([\boldsymbol{x}; \boldsymbol{h}])) + 0.1$: adaptive time constant

Euler discretization:

$$\boldsymbol{h}_{t+1} = \boldsymbol{h}_t + \Delta t \cdot \frac{-\boldsymbol{h}_t + f(\boldsymbol{x}_t, \boldsymbol{h}_t)}{\tau(\boldsymbol{x}_t, \boldsymbol{h}_t)} \tag{11}$$

프로젝트에서는 SingleStep 모드를 사용한다 — Mamba의 최종 hidden state에 대해 1회 ODE step만 적용. Mamba가 이미 $O(L)$에서 전체 시퀀스 패턴을 포착하므로 LNN은 time-scale correction만 추가한다.

금융 해석: 금융 거래 간격은 극도로 불규칙하다 (일내 다수 거래: $\Delta t \sim 0.01$일, 주말 공백: $\Delta t = 2$일, 장기 휴면: $\Delta t > 30$일). Adaptive τ가 자동으로 조정한다.

PatchTST (Patch Time Series Transformer)

Patch embedding:

$$\boldsymbol{p}_i = \boldsymbol{W}_{\text{proj}} \cdot \text{flatten}\Big(\boldsymbol{x}_{[[(i-1)P+1:iP]]}\Big) + \boldsymbol{b}_{\text{proj}} \tag{12}$$

$P = 16$으로 180-step 시퀀스가 12개 패치(토큰)가 되어 attention 비용이 $O(180^2) \rightarrow O(12^2) = 144$로 감소한다.

금융 해석: 패치 크기 16은 $\sim$2주에 해당하며 급여 주기(격주/월)와 자연스럽게 정렬된다. 각 패치 내에서 지역 패턴(2주 내 일일 지출)을, 패치 간 attention으로 전역 주기성(월급 급등, 분기 보너스)을 포착한다.

Ensemble Gating

$$\boldsymbol{g} = \text{Softmax}(\boldsymbol{W}_2 \cdot \text{ReLU}(\boldsymbol{W}_1 \cdot \boldsymbol{z}_{\text{cat}} + \boldsymbol{b}_1) + \boldsymbol{b}_2) \in \mathbb{R}^3 \tag{13}$$

$$\boldsymbol{y} = \sum_{i=1}^{3} g_i \cdot \text{Proj}_i(\boldsymbol{z}_i) \in \mathbb{R}^{64} \tag{14}$$

$\boldsymbol{z}_{\text{cat}}$는 세 모델 출력의 연결 (192+96+96 = 384D). Gate entropy $H(\boldsymbol{g}) = -\sum_{i=1}^{3} g_i \log_2(g_i)$를 모니터링하여 $H < 0.3$ bits일 경우 gate collapse를 감지한다.

금융 적용 (Financial Interpretation)

금융 거래는 강한 주기성(월급일, 주말 외식), 점진적 추세(라이프스타일 변화, 이탈 접근), 불규칙 잔차(여행, 사기)를 보인다. 각 구성요소는 모델의 강점에 직접 매핑된다: Mamba $\rightarrow$ 추세, PatchTST $\rightarrow$ 계절성, LNN $\rightarrow$ 잔차.

입출력 사양

Input	Transaction seq $[B, 180, 16]$ + Session seq $[B, 90, 8]$
Mamba output	128D + 64D = 192D
LNN output	64D + 32D = 96D (SingleStep on Mamba final state)
PatchTST output	64D + 32D = 96D
Ensemble output	64D (gated combination for PLE gate)

Table 5: Temporal Ensemble Expert 입출력 사양.

구현 참고사항

- Mamba $\rightarrow$ LNN은 serial (Mamba 학습 후 LNN이 time-scale 보정), PatchTST는 independent. 이는 input 분리를 통한 앙상블 다양성 보장 설계이다.
- A 행렬은 HiPPO-style 대각선 $[-1, -2, ..., -N]$으로 초기화하여 다중 스케일 메모리 감쇠를 구현한다.
- Gate entropy가 $\log_2(3) \approx 1.585$ bits (균일 분포)에서 < 0.3 bits로 떨어지면 한 모델이 지배하는 것으로 gate collapse 경고를 발생시킨다.

주요 참고문헌: Gu & Dao (NeurIPS 2023), Hasani et al. (AAAI 2021), Nie et al. (ICLR 2023).

Hyperbolic GCN Expert — Hierarchical Structure

선정 근거 (Why Hyperbolic GCN?)

추천 데이터는 두 가지 근본적으로 다른 기하학적 구조를 가진다: (1) 사용자-아이템 상호작용은 peer-to-peer (계층 없음), (2) 가맹점 분류 코드(MCC)는 트리 계층구조 (Root → L1 → L2 → Brand → Branch, ∼550K 노드). 단일 기하학으로는 둘 다 효율적으로 표현할 수 없다.

깊이 d의 완전 이진 트리를 유클리드 공간에 등간격으로 임베딩하려면 $O(2^d)$ 차원이 필요하다. ∼50K 브랜드 수준 노드를 왜곡 없이 임베딩하려면 수만 차원이 필요하다. 쌍곡 공간(음의 곡률)은 원점에서 멀어질수록 지수적으로 체적이 증가하여 트리 분기와 정확히 일치한다. **8D Poincare Ball로 전체 MCC 계층구조를 표현할 수 있다.**

Nickel & Kiela (2017)는 WordNet 계층구조에서 5D 쌍곡 임베딩이 200D 유클리드 임베딩을 능가함을 보였다.

대안 비교

- **LightGCN vs. NGCF:** feature transformation W와 비선형 활성화 σ를 제거하면 ID 기반 협업 필터링에서 오히려 성능이 향상된다. 변환할 raw feature가 없으므로 단순할수록 좋다.
- **H-GCN vs. Euclidean tree embedding:** 깊이 d 완전 이진 트리는 유클리드에서 $O(2^d)$ 차원이 필요하지만 쌍곡 공간에서는 $O(d)$ 차원이면 충분하다.
- **2-Stage vs. end-to-end GCN:** Pinterest PinSage 패턴 — 오프라인 그래프 사전 계산은 프로덕션 시스템의 표준이다. 그래프 업데이트 빈도와 모델 학습 빈도를 분리한다.

수학적 배경

Poincare Ball Model (H-GCN)

$$\mathbb{B}_c^d = \{\boldsymbol{x} \in \mathbb{R}^d : c\,\|\boldsymbol{x}\|^2 < 1\} \tag{15}$$

Exponential map (tangent → hyperbolic):

$$\exp_{\mathbf{0}}(\boldsymbol{v}) = \tanh\!\left(\sqrt{c}\,\|\boldsymbol{v}\|\right) \cdot \frac{\boldsymbol{v}}{\sqrt{c}\,\|\boldsymbol{v}\|} \tag{16}$$

Logarithmic map (hyperbolic → tangent):

$$\log_{\mathbf{0}}(\boldsymbol{y}) = \operatorname{arctanh}\!\left(\sqrt{c}\,\|\boldsymbol{y}\|\right) \cdot \frac{\boldsymbol{y}}{\sqrt{c}\,\|\boldsymbol{y}\|} \tag{17}$$

Poincare distance:

$$d_{\mathbb{B}}(\boldsymbol{x}, \boldsymbol{y}) = \frac{1}{\sqrt{c}} \operatorname{arccosh}\left(1 + \frac{2c\,\|\boldsymbol{x} - \boldsymbol{y}\|^2}{(1 - c\|\boldsymbol{x}\|^2)(1 - c\|\boldsymbol{y}\|^2)}\right) \tag{18}$$

경계 근처: 분모 $\to 0$이므로 거리가 폭발한다 (작은 유클리드 거리 = 큰 쌍곡 거리). 이것이 계층적 깊이를 자연스럽게 인코딩한다.

Riemannian gradient correction:

$$\nabla_{\text{Riem}} f(\boldsymbol{x}) = \frac{\left(1 - c\|\boldsymbol{x}\|^2\right)^2}{4} \nabla_{\text{Euclid}} f(\boldsymbol{x}) \tag{19}$$

Fermi-Dirac decoder (link prediction):

$$P(\text{edge} \mid u, v) = \frac{1}{\exp((d_{\mathbb{B}}(u, v) - r)/t) + 1} \tag{20}$$

통계 물리학에서 차용. r = margin (Fermi energy), t = temperature.

Frechet mean (Einstein midpoint approximation):

$$\gamma_i = \frac{1}{\sqrt{1 - c\|\boldsymbol{x}_i\|^2}}, \quad \bar{\boldsymbol{x}} = \text{proj}\left(\frac{\sum_i w_i \gamma_i \boldsymbol{x}_i}{\sum_i w_i \gamma_i}\right) \tag{21}$$

Lorentz factor γ_i는 경계 포인트(전문화된 소비자)에 높은 가중치를 부여한다.

금융 적용

- **LightGCN**: 다중 hop 협업 시그널로 간접 선호를 포착. "고객 A가 스타벅스에서 구매, 고객 B도 스타벅스와 이디야에서 구매" → A가 이디야를 선호할 수 있음.
- **H-GCN**: MCC 분류 계층(Root → L1(8) → L2(∼100) → Brand(∼50K) → Branch(∼500K))은 본질적으로 트리 구조이다.
- **Co-visitation edges**: 행동 시그널이 정적 MCC 계층을 보완한다. "스타벅스 방문자가 7일 내 이디야도 방문" → 지수 시간 감쇠로 가중된 에지 생성.

입출력 사양

Property	LightGCN	H-GCN
Nodes	Customers + Merchants (bipartite)	Merchants only (MCC tree)
Edges	Customer-Merchant transactions	Parent-child + Brand co-visitation
Space	Euclidean $\mathbb{R}^{64}$	Poincare Ball $\mathbb{B}^8$
Learning	"Who likes what" (CF)	"How merchants relate" (hierarchy)
Output	Customer embedding 64D (direct)	Merchant emb → per-customer agg 47D

Table 6: Dual GCN 아키텍처 비교.

구현 참고사항

- **2-Stage pipeline:** Stage 1 (offline) — 그래프 레벨 학습 (LightGCN: BPR, H-GCN: self-supervised Fermi-Dirac). 임베딩은 Parquet으로 저장. Stage 2 (online) — lookup + lightweight MLP adaptation. 추론 시 그래프 전파 없음 — 단일 GPU VRAM 친화적.
- Co-visitation 에지의 스케일 팩터 0.5를 적용하여 taxonomy 에지의 구조적 우위를 보존한다.

주요 참고문헌: He et al. (SIGIR 2020), Chami et al. (NeurIPS 2019), Nickel & Kiela (NeurIPS 2017).

PersLay / TDA Expert — Topological Structure

선정 근거 (Why TDA/PersLay?)

전통적 통계(평균, 분산, 상관)는 분포의 모멘트를 요약하지만 데이터의 구조적 형태 — 클러스터 연결성, 순환적 소비 패턴, 소비 공간의 void — 를 놓친다. 평균/분산이 동일한 두 고객이 근본적으로 다른 소비 위상을 가질 수 있다 (하나의 연속 클러스터 vs. 주기적 전환이 있는 두 분리된 클러스터).

Persistent Homology는 모든 스케일에서 데이터를 동시에 관찰하며(filtration), 위상적 특징(연결 컴포넌트 H_0, 루프 H_1, void H_2)이 나타나고 사라지는 것을 추적한다. 세 가지 고유한 속성을 제공한다: (1) 좌표 불변성, (2) 단일 임계값 선택 없는 다중 해상도 분석, (3) stability theorem에 의한 수학적으로 보장된 노이즈 강건성.

대안 비교

Method	Variable-size PD	End-to-end learnable	Stability guarantee
Persistence Images	Fixed grid	No	Approximate
Persistence Landscapes	Function space	No (Banach space)	Yes
PersLay	**Yes (phi + rho)**	**Yes**	**Yes (inherits)**
Persformer (2024)	Yes (attention)	Yes	Yes

Table 7: PD 처리 방법 비교. PersLay는 sum aggregation으로 gradient bottleneck이 없고 persistence weighting으로 패딩 자동 처리가 가능하여 프로덕션 안정성이 높다.

수학적 배경

PersLay Layer (Carriere et al., 2020)

$$\mathrm{PersLay}(D) = \rho\left(\sum_{(b,d)\in D} w(b,d)\cdot\varphi(b,d)\right) \tag{22}$$

- φ: point transformation (RationalHat or Gaussian)
- w: weighting function (persistence-based: $w = |d-b|^p$, or learned)
- ρ: permutation-invariant aggregation (sum, mean, max, or attention)

Homology Groups

$$H_k = \mathrm{Ker}(\partial_k)/\ \mathrm{Im}(\partial_{k+1}) \tag{23}$$

$\beta_k = \mathrm{rank}(H_k)$: 독립적인 k-차원 "구멍"의 수. β_0 = connected components, β_1 = loops, β_2 = voids.

Stability Theorem

$$d_B(\mathrm{Dgm}(f), \mathrm{Dgm}(g)) \le \|f-g\|_\infty \tag{24}$$

입력 섭동이 filtration 함수의 최대 변화로 바운드된다. 위상 피처의 노이즈 강건성을 수학적으로 보장한다.

Vietoris-Rips Complex

$$\sigma = \{x_0, ..., x_k\} \in \mathrm{VR}_\varepsilon(X) \Longleftrightarrow d(x_i, x_j) \leq \varepsilon, \quad \forall i, j \tag{25}$$

Persistence Entropy (Rucco et al., 2016)

$$E = -\sum_{i=1}^{N} p_i \log p_i, \quad p_i = \frac{d_i - b_i}{\sum_j (d_j - b_j)} \tag{26}$$

높은 entropy = 균일하게 분포된 다양한 위상 피처. 낮은 entropy = 소수의 지배적 패턴.

Wasserstein-1 Distance for Phase Transition

$$W_1(\mathrm{PD}_1, \mathrm{PD}_2) = \inf_\gamma \sum_{x \in \mathrm{PD}_1} \|x - \gamma(x)\|_\infty \tag{27}$$

전반부와 후반부 거래의 persistence diagram 간 구조적 변화를 측정한다.

Phase transition probability:

$$P_{\text{transition}} = \frac{1}{1 + e^{-2(\Delta_{\text{total}} - \tau)}}, \quad \tau = 0.5 \tag{28}$$

5-Block Multi-Beta Architecture

Block	Input	Homology	Output
short_beta0	90-day app logs $[B, 200, 3]$	H_0 (clusters)	64D
short_beta1	90-day app logs	H_1 (loops)	64D
long_beta0	12-month txn $[B, 150, 3]$	H_0 (clusters)	64D
long_beta1	12-month transactions	H_1 (loops)	64D
long_beta2	12-month transactions	H_2 (voids)	64D

Table 8: PersLay 5-block 구성.

Short concat 128D + Long concat 192D + Global stats MLP 32D + Phase transition 10D = **362D → final_mlp → 64D**.

TDA Offline Feature Pipeline (70D)

70D = 24D (short) + 36D (long) + 10D (phase):

- **tda_short (24D):** 90일 앱 로그. 6D point cloud per transaction. $H_0 + H_1 \times 6$ features (entropy, lifetime mean/std/min/max/median) $\times 2$ scopes.
- **tda_long (36D):** 12개월 카드 거래. $H_0 + H_1 + H_2 \times 6$ features $\times 2$ scopes.
- **phase_transition (10D):** PD Distance 4D + Transition Detection 6D.

금융 적용

- **H_0 (connected components):** 분리된 소비 클러스터가 라이프스타일 세분화를 드러낸다. "식료품 클러스터" vs "여행 클러스터"가 다른 스케일에서 합쳐지는 것은 소비 다양화를 나타낸다.
- **H_1 (loops):** 순환적 소비 패턴(월간: 식료품 → 교통 → 오락 → 식료품)이 persistent loop로 포착된다. 강한 loop = 습관적 패턴 = 예측 가능한 행동.
- **H_2 (voids):** 금액-카테고리-시간 공간의 3D void는 체계적 회피 패턴을 드러낸다 (예: 중간 규모 지출 부재 = 소액 + 대액만).

- **Phase transition:** Wasserstein distance로 행동 변화를 정량화한다 (이직, 생애 이벤트, 재정 위기).

입출력 사양

PersLay Input	Raw persistence diagrams (dual mode: raw PD / pre-computed 70D fallback)
PersLay Output	64D expert representation for PLE gate
TDA Offline Output	70D features integrated into main 734D tensor
Computation	Ripser++ (GPU) $\rightarrow$ Ripser (CPU) $\rightarrow$ giotto-tda (fallback)

Table 9: PersLay / TDA Expert 입출력 사양.

구현 참고사항

- Production config: RationalHatPhi + persistence weighting + sum aggregation.
- Cold-start 4-stage progressive TDA: Day 0 (18D median) $\rightarrow$ 7–30d (9D histogram) $\rightarrow$ <12m (24D H0,H1) $\rightarrow$ 12m+ (36D full).
- Time-stratified sampling: 고객당 최대 1000 포인트, k개 시간 버킷에 걸쳐 층화 추출하여 시간 순서를 보존한다.

주요 참고문헌: Carriere et al. (AISTATS 2020), Cohen-Steiner et al. (DCG 2007), Bauer (2021).

LightGCN Expert — Collaborative Filtering

선정 근거 (Why LightGCN?)

추천 시스템에서 사용자-아이템 상호작용 그래프는 peer-to-peer 구조(계층 없음)를 가진다. 이 구조에서의 핵심 질문은 "누가 무엇을 좋아하는가"이며, 다중 hop 협업 시그널을 통해 직접 상호작용하지 않은 아이템에 대한 간접 선호를 추론해야 한다.

LightGCN은 NGCF에서 feature transformation W와 nonlinear activation σ를 제거하여 ID 기반 협업 필터링에서 오히려 성능을 향상시킨 아키텍처이다. 변환할 raw feature가 없는 상황에서 단순할수록 좋다는 원칙을 따른다.

대안 비교

- **NGCF:** feature transformation + nonlinearity 포함. 협업 필터링에서 불필요한 복잡성.
- **GraphSAGE:** 이웃 샘플링 기반으로 inductive 학습에 적합하지만, transductive 설정에서 LightGCN의 전체 이웃 활용이 우수하다.
- **GAT:** attention 메커니즘이 과적합 위험을 증가시키며, 비용 대비 이득이 미미하다.

수학적 배경

Message Passing

$$\boldsymbol{e}_u^{(k+1)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u|} \cdot \sqrt{|\mathcal{N}_i|}} \cdot \boldsymbol{e}_i^{(k)} \tag{29}$$

대칭 정규화 $\tilde{A} = D^{-1/2} A D^{-1/2}$는 발신자(인기 아이템)와 수신자 영향을 모두 감쇠시킨다.

Layer combination:

$$\boldsymbol{e}_u^{\text{final}} = \frac{1}{L+1} \sum_{k=0}^{L} \boldsymbol{e}_u^{(k)} \tag{30}$$

모든 hop(0-hop self + 1,2,3-hop neighbor)에 대한 균일 평균. 학습 가능한 attention weight 없이도 경험적으로 우수하며 과적합을 방지한다.

BPR Loss:

$$\mathcal{L}_{\text{BPR}} = -\sum_{(u,i^+,i^-)} \log \sigma(\hat{y}_{ui^+} - \hat{y}_{ui^-}) + \lambda \, \|\Theta\|^2 \tag{31}$$

금융 적용

- **Cross-selling:** "고객 A가 스타벅스에서 구매, 고객 B도 스타벅스와 이디야에서 구매" → A가 이디야를 선호할 수 있음. 은행업에서의 교차 판매에 핵심적이다.
- **Cold-start 완화:** sparse한 사용자도 다중 hop을 통해 유사 사용자의 풍부한 시그널을 활용할 수 있다.
- **확장성:** 추론 시 그래프 전파 없이 사전 계산된 임베딩을 lookup하므로 실시간 서빙에 적합하다.

입출력 사양

Input	User-item bipartite graph (customer-merchant transactions)
Embedding dim	64D (Euclidean $\mathbb{R}^{64}$)
Layers	3 hops with uniform averaging
Loss	BPR (pairwise ranking)
Output	Customer embedding 64D for PLE gate

Table 10: LightGCN Expert 입출력 사양.

구현 참고사항

- 2-Stage pipeline의 Stage 1에서 오프라인으로 BPR 학습 후 임베딩을 Parquet으로 저장한다.
- L2 정규화는 초기 임베딩에만 적용하고 GCN 출력에는 적용하지 않는다.
- H-GCN과 별도 전문가로 분리하여 유클리드(CF)와 쌍곡(계층) 기하학의 독립적 gradient flow를 보장한다.

주요 참고문헌: He et al. (SIGIR 2020), Rendle et al. (UAI 2009), Kipf & Welling (ICLR 2017).

Causal Expert (NOTEARS) — Causal Inference

선정 근거 (Why Causal Expert?)

표준 추천 시스템은 상관관계("A를 산 고객이 B도 산다")에 의존하며, 허위 연관(spurious association)과 진정한 인과 효과를 혼동한다. 예: "프리미엄 카드 소지자의 여행 보험 가입률이 높다"는 소득 수준이라는 교란 변수(confounder)에 의한 것일 수 있다 — 카드가 보험 가입을 야기하는 것이 아니다.

A/B 테스트는 gold standard이지만 규모 확장이 불가능하다 (18개 태스크 $\times$ N개 전략 = 실행 불가), 느리고 (수 주 소요), 집단 수준 ATE만 제공한다.

대안 비교

- **A/B Testing:** Gold standard이지만 규모 확장 불가, 개인 수준 ITE 불가.
- **GES/PC Algorithm:** 빈도주의 인과 구조 학습이지만 조건부 독립 검정의 통계적 검정력 이슈.
- **DoWhy/EconML:** 인과 효과 추정에 특화되지만 DAG 학습은 별도.
- **DAGMA (Bello et al., 2022):** NOTEARS 개선판으로 향후 교체 가능한 후보.

수학적 배경

Feature Compression

$$\boldsymbol{z} = \text{Compressor}(\boldsymbol{x}) : \mathbb{R}^{644} \to \mathbb{R}^{128} \to \mathbb{R}^{32} \tag{32}$$

644D 정규화 피처를 32개 인과 변수로 축소한다. DAG 인접 행렬이 $644^2 \approx 410$K 엔트리로 폭발하는 것을 방지한다.

SCM (Structural Causal Model) Intervention

$$\hat{\boldsymbol{z}} = \boldsymbol{z} + \boldsymbol{z}(\boldsymbol{W} \circ \boldsymbol{W}) \tag{33}$$

- $\boldsymbol{W} \in \mathbb{R}^{32\times 32}$: 학습 가능한 가중 인접 행렬 (`nn.Parameter`)
- $\boldsymbol{W} \circ \boldsymbol{W}$: element-wise 제곱 $\to$ 비음수 인과 강도 보장
- $W_{i,j}^2$: 변수 j에서 i로의 인과 영향 강도
- Residual connection ($\boldsymbol{z} +$)이 원본 정보를 보존하면서 인과 조정을 추가

NOTEARS Acyclicity Constraint

$$h(\boldsymbol{W}) = \text{tr}(e^{\boldsymbol{W} \circ \boldsymbol{W}}) - d = 0 \tag{34}$$

수학적 해석: $e^{\boldsymbol{M}}$의 (i, i) 대각 원소는 노드 i에서 자신으로 돌아오는 모든 가중 경로의 합이다. 그래프가 DAG(비순환)이면 이러한 회귀 경로가 존재하지 않으므로 $e_{i,i}^{\boldsymbol{M}} = 1$ (항등 행렬 기여분만), $\text{tr}(e^{\boldsymbol{M}}) = d$가 된다.

Taylor 10-term approximation:

$$e^{\boldsymbol{M}} \approx \sum_{k=0}^{9} \frac{\boldsymbol{M}^k}{k!} \tag{35}$$

길이 10까지의 cycle을 감지한다. 32-node DAG에서 10-hop 이상의 cycle은 실질적으로 불가능하다.

DAG Regularization Loss

$$\mathcal{L}_{\text{DAG}} = \lambda_{\text{acyclic}} \cdot h(\boldsymbol{W}) + \lambda_{\text{sparse}} \cdot \|\boldsymbol{W} \circ \boldsymbol{W}\|_1 \tag{36}$$

Hyperparameter	Default	Role
`dag_lambda`	0.01	Acyclicity constraint strength
`sparsity_lambda`	0.001	Edge sparsity (L1 on adjacency)
`n_causal_vars`	32	Number of causal nodes

Table 11: Causal Expert 하이퍼파라미터. `dag_lambda > 0.1`이면 W가 영행렬로 붕괴한다 (DAG 페널티가 태스크 손실을 지배, Expert가 항등 함수로 퇴화).

금융 적용

- 교란 변수(income)를 제거하고 방향성 인과 관계를 학습하여 "이 추천이 행동 변화를 야기할 것인가?"에 답한다.
- SCM의 방향성($W_{i,j} \neq W_{j,i}$)이 설명 가능한 추천 경로를 제공한다.
- 관찰 데이터에서 individual treatment effect (ITE) 추정을 가능하게 한다.

입출력 사양

Input	644D normalized features $[B, 644]$
Compressor	$644 \to 128 \to 32$ (causal variables)
SCM	32×32 learnable adjacency $\boldsymbol{W}$
Output	64D causal representation + DAG (visualization)
Auxiliary loss	$\mathcal{L}_{\text{DAG}} = \lambda_{\text{acyclic}} \cdot h(\boldsymbol{W}) + \lambda_{\text{sparse}} \cdot \|\boldsymbol{W}^2\|_1$

Table 12: Causal Expert 입출력 사양.

구현 참고사항

- NOTEARS 논문은 Augmented Lagrangian (엄격한 등식)을 사용하지만 구현에서는 simple penalty method를 사용한다. End-to-end MTL joint training 환경에서 더 안정적이다.
- 인접 행렬 $\boldsymbol{W}$의 Hadamard square ($\boldsymbol{W} \circ \boldsymbol{W}$)는 sign-agnostic 원본과 달리 비음수 인과 강도를 강제한다.
- OT Expert와 별도 전문가로 유지한다: NOTEARS acyclicity constraint와 Sinkhorn entropy regularization의 loss surface 기하학이 완전히 다르다.

주요 참고문헌: Zheng et al. (NeurIPS 2018), Bello et al. (ICML 2022).

Optimal Transport Expert — Distributional Matching

선정 근거 (Why Optimal Transport?)

고객의 소비 패턴을 프로토타입 프로필과 비교할 때 피처 공간의 기하학적 구조를 존중해야 한다. KL divergence는 분포의 support가 겹치지 않으면 정의되지 않고, 유클리드 거리는 피처 공간의 기하학적 구조를 무시한다. Wasserstein distance는 ground metric space의 기하학을 반영한다 — "서울–인천"이 "서울–부산"보다 가깝다는 것을 확률 질량이 겹치지 않아도 포착한다.

Causal Expert와 상보적: Causal은 "이 추천이 행동 변화를 야기할 것인가?" (방향성), OT는 "이 고객의 소비 분포가 목표 프로필에 얼마나 가까운가?" (기하학적 거리).

대안 비교

- **KL Divergence:** support가 겹치지 않으면 undefined. 비대칭.
- **Total Variation:** ground metric 무시, 분포 형태만 비교.
- **Euclidean distance:** 피처 공간의 의미적 구조 무시.
- **Sliced Wasserstein:** 계산 효율적 대안, 향후 독립적으로 교체 가능.

수학적 배경

Distribution Projection

$$\boldsymbol{\mu} = \mathrm{softmax}(\mathrm{DistProjector}(\boldsymbol{x})) \in \Delta^{32} \tag{37}$$

644D 피처를 확률 심플렉스로 변환하여 각 고객의 피처 프로필을 32개 잠재 카테고리에 대한 이산 분포로 표현한다.

Learnable Reference Distributions

$$\boldsymbol{\nu}_k = \mathrm{softmax}(\ell_k) \in \Delta^{32}, \quad k = 1, \ldots, 16 \tag{38}$$

16개의 학습 가능한 프로토타입 고객 프로필 (`nn.Parameter`), `randn(16, 32) * 0.1`으로 초기화.

PSD Cost Matrix

$$\boldsymbol{C} = \boldsymbol{M}^\top \boldsymbol{M} \in \mathbb{R}^{32\times 32} \tag{39}$$

양의 준정부호성(PSD) 보장: $\boldsymbol{x}^\top(\boldsymbol{M}^\top\boldsymbol{M})\boldsymbol{x} = \|\boldsymbol{M}\boldsymbol{x}\|^2 \geq 0$. 비용 행렬이 학습 가능하여 태스크에 최적화된 시맨틱 거리를 제공한다.

Entropy-Regularized Optimal Transport

Kantorovich problem with entropic regularization:

$$\min_{\boldsymbol{P}\in\mathcal{U}(\boldsymbol{\mu},\boldsymbol{\nu})} \langle \boldsymbol{P}, \boldsymbol{C}\rangle + \varepsilon \cdot H(\boldsymbol{P}) \tag{40}$$

여기서:

- $\mathcal{U}(\boldsymbol{\mu}, \boldsymbol{\nu}) = \{\boldsymbol{P} \geq 0 : \boldsymbol{P}\mathbf{1} = \boldsymbol{\mu}, \boldsymbol{P}^\top \mathbf{1} = \boldsymbol{\nu}\}$
- $H(\boldsymbol{P}) = -\sum_{i,j} P_{i,j} \log P_{i,j}$: entropy regularization
- $\varepsilon = 0.1$: regularization coefficient

Entropy regularization은 문제를 **strictly convex**로 만든다 (유일 해, 수렴 보장).

Log-Domain Sinkhorn Algorithm

$$\boldsymbol{u}_{\text{new}} = \log \boldsymbol{\mu} - \text{logsumexp}\left(-\frac{\boldsymbol{C}}{\varepsilon} + \boldsymbol{v}\right) \tag{41}$$

$$\boldsymbol{v}_{\text{new}} = \log \boldsymbol{\nu} - \text{logsumexp}\left(-\frac{\boldsymbol{C}^\top}{\varepsilon} + \boldsymbol{u}\right) \tag{42}$$

Log-domain 계산으로 ε이 작을 때의 floating-point underflow를 방지한다. 10회 반복이면 실용적 수렴에 충분하다.

Wasserstein Distance Vector

$$\boldsymbol{w} = [W(\boldsymbol{\mu}, \boldsymbol{\nu}_1), W(\boldsymbol{\mu}, \boldsymbol{\nu}_2), ..., W(\boldsymbol{\mu}, \boldsymbol{\nu}_{16})] \in \mathbb{R}^{16} \tag{43}$$

여기서 $W(\boldsymbol{\mu}, \boldsymbol{\nu}_k) = \langle \boldsymbol{P}, \boldsymbol{C} \rangle_F = \sum_{i,j} P_{i,j} \cdot C_{i,j}$.

이것은 **distributional coordinate system**을 생성한다 — 각 고객이 16개 참조 프로토타입까지의 거리로 위치가 결정된다.

Wasserstein Encoder

$$\boldsymbol{o} = \text{WassersteinEncoder}(\boldsymbol{w}) : \mathbb{R}^{16} \to \mathbb{R}^{128} \to \mathbb{R}^{64} \tag{44}$$

금융 적용

Wasserstein distance는 "이 고객의 소비 패턴이 전형적인 여행형/저축형/외식형 프로필과 얼마나 다른가, 그리고 어떤 카테고리가 어느 방향으로 이동해야 매칭되는가?"를 정량화한다.

이는 KL divergence나 유클리드 거리로는 불가능한 방향적(directional) 정보를 제공한다.

입출력 사양

Input	644D normalized features $[B, 644]$
Distribution	$644 \to 32$ probability simplex Δ^{32}
References	16 learnable prototypes $\in \Delta^{32}$
Cost matrix	Learnable PSD: $\boldsymbol{M}^\top \boldsymbol{M} \in \mathbb{R}^{32 \times 32}$
Sinkhorn	10 iterations, log-domain, $\varepsilon = 0.1$
Output	64D expert representation for PLE gate

Table 13: Optimal Transport Expert 입출력 사양.

구현 참고사항

- Cuturi (2013) 원본은 고정 비용 행렬과 단일 target을 사용하지만, 구현에서는 학습 가능한 PSD 비용 행렬과 16개 학습 가능 프로토타입을 사용한다.
- Sinkhorn 반복 횟수를 10으로 고정하여 학습 루프에서의 계산 비용을 예측 가능하게 한다.

- Causal Expert와의 synergy: DeepFM은 대칭적 피처 상호작용 $\langle \boldsymbol{v}_i, \boldsymbol{v}_j \rangle$, Causal은 비대칭 방향적 인과 $W_{i,j}^2$, OT는 거리 함수(metric) $W(\mu, \nu_k)$를 추출한다. 세 전문가는 동일한 644D 입력에서 수학적으로 완전히 다른 구조를 추출한다.

주요 참고문헌: Cuturi (NeurIPS 2013), Kantorovich (1942).

adaTT (Adaptive Task-aware Transfer)

동기: Multi-Task Learning에서의 Negative Transfer

18개 동시 태스크가 전문가 파라미터를 공유할 때 gradient 충돌이 negative transfer를 야기한다. 고정 타워 MTL의 세 가지 근본적 한계: (1) 공유 backbone이 모든 태스크에 동일하게 영향 — 한 태스크의 최적화가 다른 태스크 예측을 악화시켜도 감지/방지 메커니즘 없음, (2) 18개 태스크 간 어떤 쌍이 서로 돕는지/해치는지 측정 불가, (3) 고정 가중치로는 학습 단계에 따라 변하는 태스크 관계를 추적 불가.

Core Mechanism: Gradient Cosine Similarity

$$\cos(\theta_{i,j}) = \frac{\boldsymbol{g}_i \cdot \boldsymbol{g}_j}{\|\boldsymbol{g}_i\| \cdot \|\boldsymbol{g}_j\|} \tag{45}$$

여기서 $\boldsymbol{g}_i = \nabla_\theta \mathcal{L}_i$는 태스크 i의 손실에 대한 공유 전문가 파라미터의 gradient.

왜 cosine인가 (유클리드 아님): (1) 스케일 불변 — 태스크 손실이 수 자릿수 차이나도 방향만 비교, (2) 해석 가능한 범위 $[-1, 1] \rightarrow$ positive/negative transfer에 직접 매핑, (3) 효율적 계산: 정규화 후 단일 행렬 곱 $\hat{\boldsymbol{G}}\hat{\boldsymbol{G}}^\top$으로 모든 n^2 유사도 계산.

EMA Stabilization

$$\boldsymbol{A}_t = \alpha \cdot \boldsymbol{A}_{t-1} + (1-\alpha) \cdot \cos(\theta_t) \tag{46}$$

$\alpha = 0.9$ (유효 윈도우 ≈ 10 관측). IIR 1차 저역통과 필터와 동등하며 고주파 배치 노이즈를 제거하면서 진정한 태스크 관계 추세를 보존한다.

Transfer-Enhanced Loss

$$\mathcal{L}_i^{\text{adaTT}} = \mathcal{L}_i + \lambda \cdot \sum_{j \neq i} w_{i \rightarrow j} \cdot \mathcal{L}_j \tag{47}$$

$\lambda = 0.1$ (다른 태스크로부터 10% 영향), `max_transfer_ratio = 0.5` (전이 손실이 원래 손실의 50%를 초과할 수 없음).

Gradient 영향:

$$\nabla_\theta \mathcal{L}_i^{\text{adaTT}} = \nabla_\theta \mathcal{L}_i + \lambda \sum_{j \neq i} w_{i \rightarrow j} \nabla_\theta \mathcal{L}_j \tag{48}$$

두 번째 항은 다중 태스크에 유익한 방향으로 공유 파라미터를 조정하는 보정 벡터이다.

Transfer Weight Computation (4-stage)

$$\boldsymbol{R} = (\boldsymbol{W} + \boldsymbol{A}) \cdot (1-r) + \boldsymbol{P} \cdot r \tag{49}$$

$$\boldsymbol{R}_{i,j} \leftarrow 0 \quad \text{if } \boldsymbol{A}_{i,j} < \tau_{\text{neg}} \tag{50}$$

$$\boldsymbol{R}_{i,i} = 0 \tag{51}$$

$$w_{i \to j} = \text{softmax}\left(\frac{\boldsymbol{R}_{i,j}}{T}\right) \tag{52}$$

- $\boldsymbol{W}$: 학습 가능한 전이 가중치 (`nn.Parameter`, 0으로 초기화)
- $\boldsymbol{A}$: EMA affinity matrix
- $\boldsymbol{P}$: Group Prior matrix (도메인 지식)
- r: Prior blend ratio ($0.5 \to 0.1$로 annealing)
- $\tau_{\text{neg}} = -0.1$: Negative transfer threshold
- $T = 1.0$: Softmax temperature

Group Prior

Group	Tasks	Intra-strength	Business Meaning
engagement	has_nba, engagement_score, cross_sell_count, will_acquire_deposits, will_acquire_investments, will_acquire_accounts, will_acquire_lending, will_acquire_payments	0.8	고객 관여/전환
lifecycle	churn_signal, product_stability, tenure_stage, segment_prediction	0.7	고객 생애주기
value	income_tier, spend_level, nba_primary	0.6	고객 가치/행동
consumption	next_mcc, mcc_diversity_trend, top_mcc_shift	0.7	소비 패턴

Table 14: adaTT 태스크 그룹 정의. Inter-group strength: 0.3.

Prior Blend Annealing (Bayesian 해석):

$$r(e) = r_{\text{start}} - (r_{\text{start}} - r_{\text{end}}) \cdot \min\left(\frac{e - e_{\text{warmup}}}{e_{\text{freeze}} - e_{\text{warmup}}}, 1.0\right) \tag{53}$$

$r : 0.5 \to 0.1$은 prior-to-posterior 전환을 구현한다: 초기 학습은 도메인 지식(prior)에 의존, 후기 학습은 관측된 gradient affinity(likelihood)를 신뢰한다.

3-Phase Schedule

Phase	Period	Behavior	Purpose
Warmup	Epoch 0 – warmup	Affinity만 계산, 전이 손실 없음	안정적 affinity 데이터 축적
Dynamic	warmup – freeze	Active transfer + annealing prior	태스크 관계 학습 및 적용
Frozen	freeze – end	고정된 전이 가중치 (detached)	fine-tuning 안정화, gradient overhead 제거

Table 15: adaTT 3-Phase 학습 스케줄.

Negative Transfer Detection

$$\boldsymbol{R}_{i,j} = \begin{cases} \boldsymbol{R}_{i,j} & \text{if } \boldsymbol{A}_{i,j} > \tau_{\text{neg}} \\ 0 & \text{otherwise} \end{cases} \tag{54}$$

$\tau_{\text{neg}} = -0.1$ (0이 아님)로 약한 음의 상관(noise 가능성)을 허용하면서 명확한 적대적 gradient는 차단한다.

Transformer Self-Attention과의 유사성

Role	Transformer	adaTT
Query	Current token's query	Current task's gradient direction
Key	Other tokens' response	Other tasks' gradient directions
Similarity	$Q\frac{K^\top}{\sqrt{d_k}}$	Gradient cosine similarity
Value	Other tokens' information	Other tasks' loss values
Output	Weighted context	Transfer loss

Table 16: adaTT와 Transformer self-attention의 구조적 유사성.

대안 비교

Method	Mechanism	adaTT 대비 한계
Fixed Weighting	Manual task weights	동적 affinity 측정 불가
GradNorm	Gradient magnitude balancing	방향이 아닌 크기만 사용
PCGrad	Project conflicting gradients	positive knowledge 선택적 전이 불가
Nash-MTL	Nash bargaining for Pareto	계산 비용 ($O(n^2d)$ vs optimization)
CAGrad	Worst-case gradient alignment	측정과 적용의 분리(modularity) 부족

Table 17: MTL 접근법 비교.

구현 참고사항

- **Phase 1 (Shared Expert Pretrain):** adaTT active — 15 epochs 동안 gradient extraction + transfer loss.
- **Phase 2 (Cluster Finetune):** adaTT disabled — Shared experts frozen, cluster-specific sub-heads만 8 epochs 학습.
- Hypernetworks (Ha et al., 2017)의 경량 변형: 학습된 task embedding 대신 관측된 gradient를 conditioning signal로 사용하여 변화하는 태스크 관계에 zero-delay 적응.
- `detect_negative_transfer()` API가 각 태스크의 적대적 태스크 목록을 반환 (예: `{"churn_signal": ["has_nba", "engagement_score"]}`).

주요 참고문헌: Tang et al. (RecSys 2020), Yu et al. (NeurIPS 2020), Fifty et al. (ICML 2021), Chen et al. (ICML 2018), Navon et al. (ICML 2022).

Feature Engineering Overview — 11 Disciplines, 316D

피처 엔지니어링 철학

전통적 통계 피처는 단일 렌즈로 데이터를 바라본다. 다른 학문 분야들은 수세기에 걸쳐 특정 패턴 유형에 최적화된 수학적 도구를 개발해왔다. 핵심 통찰은 **구조적 동형(structural isomorphism)** — 수학적 관계 구조가 도메인 객체와 무관하게 동일할 때 수식은 표면적 도메인에 관계없이 같은 패턴을 포착한다.

11개 학문 분야별 피처 분류

Discipline	Dim	Pattern	Mathematical Tool
Economics (PIH)	8D	영구/일시 소득 분해	HP Filter, Kalman Filter
Economics (Micro)	9D	탄력성, 소비 평활화	Arc elasticity, HHI, Shannon entropy
Chemical Kinetics	6D	행동 변화 속도/가속도	Arrhenius equation, finite differences
SIR Epidemiology	5D	카테고리 채택 확산	ODE compartmental model
Routine Activity (Criminology)	5D	규칙성, burstiness, 시간 이상치	Circular statistics, burstiness index
Wave Interference	8D	주기 분해, 위상 동기화	FFT, Hilbert transform, PLV
TDA (Topology)	70D	소비 위상 구조, 위상 변이	Persistent Homology, Wasserstein-1
GMM (Statistics)	22D	소프트 세분화, 불확실성	EM algorithm, BIC
HMM (Probabilistic)	48D	잠재 상태 전이, 동적 궤적	Forward-Backward, Viterbi, ODE dynamics
MAB (Decision Theory)	4D	탐색/활용 균형	HHI trend, recency-weighted entropy
Graph Embedding	111D	협업 필터링 + 계층 구조	LightGCN (64D) + H-GCN (47D)

Table 18: 11개 학문 분야에서 도출된 피처 체계. 총 316D (TDA 70D + GMM 22D + HMM 48D + Economics 17D + Multidisciplinary 24D + MAB 4D + Graph 111D + LNN statistics 18D + HMM 5D summary = 316D+ 모델 파생 피처).

Economics Features (17D)

Friedman PIH (8D)

관측 소득을 영구(permanent) 성분과 일시(transitory) 성분으로 분해한다:

$$Y_t = Y_t^P + Y_t^T \tag{55}$$

소비 함수: $C_t = k(r, w, u) \cdot Y_t^P$ — 소비자는 영구 소득에 비례하여 지출하고, 일시 소득은 저축/투자로 향한다.

세 가지 추정 방법을 지원한다:
- Moving Average: $\hat{Y}_t^P = \frac{1}{L}\sum_{i=0}^{L-1} Y_{t-i}$, $L = 12$
- HP Filter: $\min_\tau \left\{\sum_t (Y_t - \tau_t)^2 + \lambda \sum_t \left[(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1})\right]^2\right\}$, $\lambda = 14400$
- Kalman Filter: $K_{\text{ss}} \approx 0.27$ (73% prior weight, 27% observation weight)

Microeconomic Behavior (9D)

Income Elasticity:

$$\varepsilon_Y = \frac{\partial Q}{\partial Y} \cdot \frac{Y}{Q} = \frac{d \ln Q}{d \ln Y} \tag{56}$$

$\varepsilon_Y > 1$: 사치재 행동, $0 < \varepsilon_Y < 1$: 필수재, $\varepsilon_Y < 0$: 열등재.

Consumption Smoothing (Hall, 1978):

$$C_t = C_{t-1} + \varepsilon_t \tag{57}$$

Feature: `consumption_smoothing = mu/sigma` (소비의 Sharpe ratio 유사체).

Spending Diversification (Shannon Entropy):

$$H = -\sum_{i=1}^{N} s_i \ln(s_i) \tag{58}$$

Category Concentration (HHI):

$$\text{HHI} = \sum_i s_i^2 \tag{59}$$

Shannon entropy와 HHI는 Renyi entropy H_α의 특수 경우 ($\alpha \to 1$: Shannon, $\alpha = 2$: HHI).

Multidisciplinary Features (24D)

네 개 모듈이 데이터의 거의 직교하는 projection을 포착한다:
- **Chemical Kinetics:** 시간의 미분적 구조 (1차, 2차 도함수)
- **Epidemic Diffusion:** 상태 공간 전이 구조 ($S \to I \to R$)
- **Crime Pattern:** 시계열의 통계적 텍스처 (주기성, 클러스터링, 분산)
- **Interference:** 주파수 도메인 스펙트럼 구조 (FFT, coherence, 위상)

교차 모듈 조합으로 개별 모듈에서는 보이지 않는 패턴을 발견한다: 예: 높은 `catalyst_sensitivity` + 높은 `burstiness` = 월급일 폭발 소비자 (월 초 타겟 프로모션 최적 대상).

GMM Clustering Features (22D)

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{60}$$

$K = 20$ 클러스터, $D = 40$ 입력 차원, full covariance. 출력: 20D cluster probabilities γ_{nk} + cluster ID + cluster entropy.

K-Means 대비 GMM의 핵심 장점: `GroupTaskExpertBasket`의 20개 클러스터 sub-head 출력을 γ_{nk}로 가중 앙상블한다. 소프트 할당으로 경계 고객의 추천 품질이 향상된다.

HMM Triple-Mode Features (48D)

세 개의 병렬 Hidden Markov Model:

Mode	States	Time Scale	Target Tasks
Journey (AICRA)	5	Days/weeks	CTR, CVR
Lifecycle	5	Months/years	Churn, Retention
Behavior	6	Monthly patterns	NBA, balance_util

Table 19: HMM Triple-Mode 구성. 각 mode 16D = state probs + meta features + ODE dynamics. 총 48D.

ODE Dynamics Bridge (6D per mode): Viterbi 상태 궤적에서 추출 — velocity, acceleration, Lyapunov exponent, cycle period, attractor strength, trajectory length.

Five-Axis Feature Taxonomy

전체 시스템의 734D main tensor + 68D separate input은 5개 피처 축에 걸쳐 있다:

1. **Static/Snapshot:** demographics, account status
2. **Time-series:** Mamba/LNN-derived temporal patterns
3. **Hierarchical:** merchant hierarchy, graph embeddings
4. **Item/Product:** product interaction features
5. **Model-derived:** HMM 5D summary, Bandit 4D, LNN statistics 18D

Two-Level Ensemble Architecture

전체 아키텍처는 **2단계 앙상블**:

- **Level 1:** Temporal Expert 내에서 Mamba/LNN/PatchTST를 learned gating으로 결합
- **Level 2:** 7개 Shared Expert (PersLay, DeepFM, Temporal, LightGCN, H-GCN, Causal, OT) 전체에서 CGC Gate Attention이 태스크별 결합 수행

이 계층적 앙상블이 intra-expert diversity (시간적 다중 해상도)와 inter-expert complementarity (패턴/위상/시간/관계/인과/분포)를 모두 보장한다.

References

Bauer, U. (2021). Ripser: efficient computation of Vietoris-Rips persistence barcodes. Journal of Applied and Computational Topology.

Bello, K. et al. (2022). DAGMA: Learning DAGs via M-matrices and a Log-Determinant Acyclicity Characterization. ICML.

Carriere, M. et al. (2020). PersLay: A Neural Network Layer for Persistence Diagrams. AISTATS.

Chami, I. et al. (2019). Hyperbolic Graph Convolutional Neural Networks. NeurIPS.

Chen, Z. et al. (2018). GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks. ICML.

Cohen-Steiner, D., Edelsbrunner, H., & Harer, J. (2007). Stability of Persistence Diagrams. Discrete & Computational Geometry.

Cuturi, M. (2013). Sinkhorn Distances: Lightspeed Computation of Optimal Transport. NeurIPS.

Dempster, A., Laird, N., & Rubin, D. (1977). Maximum Likelihood from Incomplete Data via the EM Algorithm. JRSS-B.

Fifty, C. et al. (2021). Efficiently Identifying Task Groupings for Multi-Task Learning. NeurIPS.

Friedman, M. (1957). A Theory of the Consumption Function. Princeton UP.

Gu, A. & Dao, T. (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. NeurIPS.

Guo, H. et al. (2017). DeepFM: A Factorization-Machine based Neural Network for CTR Prediction. IJCAI.

Ha, D. et al. (2017). HyperNetworks. ICLR.

Hall, R. (1978). Stochastic Implications of the Life Cycle-Permanent Income Hypothesis. JPE.

Hasani, R. et al. (2021). Liquid Time-constant Networks. AAAI.

He, X. et al. (2020). LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. SIGIR.

Navon, A. et al. (2022). Multi-Task Learning as a Bargaining Game. ICML.

Nickel, M. & Kiela, D. (2017). Poincare Embeddings for Learning Hierarchical Representations. NeurIPS.

Nie, Y. et al. (2023). A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. ICLR.

Rendle, S. (2010). Factorization Machines. ICDM.

Rendle, S. et al. (2009). BPR: Bayesian Personalized Ranking from Implicit Feedback. UAI.

Tang, H. et al. (2020). Progressive Layered Extraction: A Novel Multi-Task Learning Model for Personalized Recommendations. RecSys.

Wang, R. et al. (2017). Deep & Cross Network for Ad Click Predictions. KDD.

Wang, R. et al. (2021). DCN V2: Improved Deep & Cross Network. WWW.

Yu, T. et al. (2020). Gradient Surgery for Multi-Task Learning. NeurIPS.

Zheng, X. et al. (2018). DAGs with NO TEARS: Continuous Optimization for Structure Learning. NeurIPS.